

Research and Exploration of Unix File Permissions & Extended ACLs

1. Standard Permission Bottlenecks

Architectural Limitations of the 3-Tier Model: The traditional Unix file permission system is based on a very static 3-tier model called UGO (User, Group, Others). Each file or directory has exactly one owner (User), one owning group (Group), and a list of all other users that have some access to the file/os - (Others). These permissions are set according to three aspects of each folder: Read (r), Write (w), and Execute (x).

This model results in a bottleneck, since it is too coarse to cope with large-scale complex multi-user situations where we need fine-grained models:

- **Single Group Limitation:** A file can belong only to a single group exclusively. If two different departments (e.g., "HR" and "Finance") need different levels of access to the same file, the standard model forces the administrator to create a new, artificial group containing members of both, or open permissions to "Others," which is a security risk.
- **"All or Nothing" for Others:** The "Others" bucket is a mix of everything. Giving read access to "Others" makes the file readable by all users on the system and goes against the concept of least privilege.
- **Inflexibility:** You cannot give permission to a subgroup or some list of users without modifying the group itself.

Scenario of Failure: The "Specific Individual" Problem

If we think of a situation in some academic department with a file say, `exam_questions.txt`.

- Owner: Professor A (Read/Write)
- Group: `faculty` (Read-only)
- Others: None (No Access)

The Issue: Professor A wants to give Student X (a Teaching Assistant) read permission of this file specifically to decide whether he or she made mistake.

Why Standard Permissions Fail:

1. Cannot use Group: Due to the fact that adding Student X to the faculty group would give them access to everything in that group, I cannot permit that. This is a security breach.
2. Cannot use Others: If you were to set "others" to "read" then every student and user could read all the exam questions.
3. Conclusion: The standard model does not allow permissions to be identified for a specific user (u:student_x) without unjoining global groups or losing system-level-wide security

2. Mechanism of Extended ACLs

Structure of a POSIX ACL Entry

POSIX ACLs are an extension of the basic inode metadata. The inode normally contains the UID and GID only, if ACLs are activated the file system is able to write a list in an "Extended Attribute" (xattr) that will be associated with this inode.

The form of an ACL entry is usually: **type:qualifier:permissions**

- Type: The category of the entry (e.g., **user**, **group**, **mask**, **other**).
- Qualifier: The specific name or ID (e.g., **jdoe**, **finance_team**). If empty, it refers to the file owner or owning group.
- Permissions: The access rights (**r**, **w**, **x**, or **-**).

Command-Line Implementation: **chmod** vs. **setfacl** / **getfacl**

The management of these permissions differs significantly in syntax and capability.

Feature	Standard Permissions (chmod)	Extended ACLs (setfacl / getfacl)

Scope	Changes the base 3 tiers (Owner, Group, Other).	Manages specific lists of users and groups.
Viewing	<code>ls -l</code> shows the summary (presence of <code>+</code> indicates ACL).	<code>getfacl filename</code> displays the full list of entries.
Modifying	<code>chmod g+w filename</code> (Broad application).	<code>setfacl -m u:username:rwx filename</code> (Targeted application).
Removal	Permissions are changed, not removed.	Entries can be physically removed (<code>setfacl -x</code>).

Example of setfacl: To solve the scenario in section 1, the command would simply be:

Bash

```
setfacl -m u:student_x:r exam_questions.txt
```

This grants read access to `student_x` without changing group ownership or "Others" permission.

Priority Logic (The Check Algorithm)

When the process asks for permissions to enter the file with ACLs, among those priority permissions the OS kernel verifies a list of permissions. The check terminates immediately once we find a match:

1. File Owner: Grants User Element-level permission when the requester is the file owner.
2. Named User (ACL User): If the requester is a particular user as defined in ACLs, e.g. `user:john:rw-`, those permissions take effect.

3. Owning Group / Named Group: If the requestor is a member of the owning group, or in a named group that matches an entry on the ACLs, permissions are tested.
 - *Note*: If a **Mask** exists, group and named user effective permissions are subject to the mask (bitwise AND).
4. Others: If the above conditions don't match, we use regular others permission.

It prioritizes so that your rule to deny/allow certain individuals will always trump the overall rules for a group or the public.

References

Arch Linux Wiki. (n.d.). *Access Control Lists*. Retrieved January 7, 2026, from https://wiki.archlinux.org/title/Access_Control_Lists

GeeksforGeeks. (n.d.). *Access Control Lists (ACL) in Linux*. Retrieved January 7, 2026, from <https://www.geeksforgeeks.org/linux-unix/access-control-listsacl-linux/>

Red Hat, Inc. (n.d.). *Chapter 5. Access Control Lists*. Red Hat Enterprise Linux 7 System Administrator's Guide. Retrieved January 7, 2026, from https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/system_administrators_guide/ch-access_control_lists